

Retail Sales Analytics & Forecasting

Project report | Superstore | 2014-2017

A reproducible analysis using Excel, SQLite, Power BI, pandas, matplotlib and linear regression. The objective is to identify sales opportunities, profit risks and a defensible forecasting baseline.

Sales	Profit	Profit margin
\$2,297,200.86	\$286,397.02	12.47%

Executive findings

West generates the highest sales: \$725,457.82, or 31.58% of total sales. It also leads profit at \$108,418.45.

All three top-level categories are profitable. Furniture has the weakest margin at 2.49%. Tables, Bookcases and Supplies are the loss-making sub-categories.

Sales rise from \$484,247 in 2014 to \$733,215 in 2017, despite a 2.83% decline in 2015. Sales increase 20.36% in 2017.

Every observed discount level of 30% or higher has negative aggregate profit. This is an association and does not isolate the causal effect of discounts.

A trend-plus-seasonality linear regression achieves 2017 monthly MAE of \$10,794 and R-squared of 0.681. It reduces MAE by 30.22% compared with the seasonal-naive baseline.

Scope

9,994 order lines across 5,009 orders, dated 3 January 2014 through 30 December 2017. Monetary values are assumed USD because the records describe US retail activity. The file contains historical sample data; forecasts refer to the period immediately after that history, not to 2026.

1. Data preparation and quality

Check	Outcome
Rows before / after cleaning	9,994 / 9,994
Exact duplicate rows	0
Missing source cells	0
Duplicate Row IDs	0
Repeated business rows excluding Row ID	1 pair retained because Row IDs differ
Dates	Slashes and hyphens normalized; explicit month/day/year parse
Postal Code	Preserved as a five-character text identifier

Repeated Order IDs are expected because one order can contain multiple product lines. The exact-duplicate removal rule therefore operates on the entire source row. One repeated set of business fields has different Row IDs; it is flagged and retained rather than assumed erroneous.

Required dates and numeric values are validated. The reproducible notebook stops on missing required values or invalid dates instead of imputing sales or profit. Missing optional descriptive values would become Unknown. No imputation was needed for this file. Negative profits and large transactions are retained.

Derived field	Definition
Profit Ratio	Profit / Sales; blank when Sales is zero
Delivery Days	Ship Date minus Order Date, in calendar days
Order Month	Year-month of Order Date
Aggregate margin	SUM(Profit) / SUM(Sales)

Delivery Days is the project field name. The source provides a shipping date, not proof of customer delivery, so the metric measures order-to-ship time.

Excel implementation

The workbook includes an overview, formula-driven analysis, cleaned data and quality notes. Profit Ratio and Delivery Days are formulas. Negative profit uses conditional formatting. Two native pivot tables summarize regional and category sales/profit on the Analysis sheet. Refresh the pivots after changes. Existing summary ranges cover the supplied 9,994 lines; extend them when adding rows.

2. Sales and profitability

Region	Sales	Profit	Margin
West	\$725,457.82	\$108,418.45	14.94%
East	\$678,781.24	\$91,522.78	13.48%
Central	\$501,239.89	\$39,706.36	7.92%
South	\$391,721.91	\$46,749.43	11.93%

Category	Sales	Profit	Margin
Technology	\$836,154.03	\$145,454.95	17.40%
Furniture	\$741,999.80	\$18,451.27	2.49%
Office Supplies	\$719,047.03	\$122,490.80	17.04%

Loss-making sub-categories

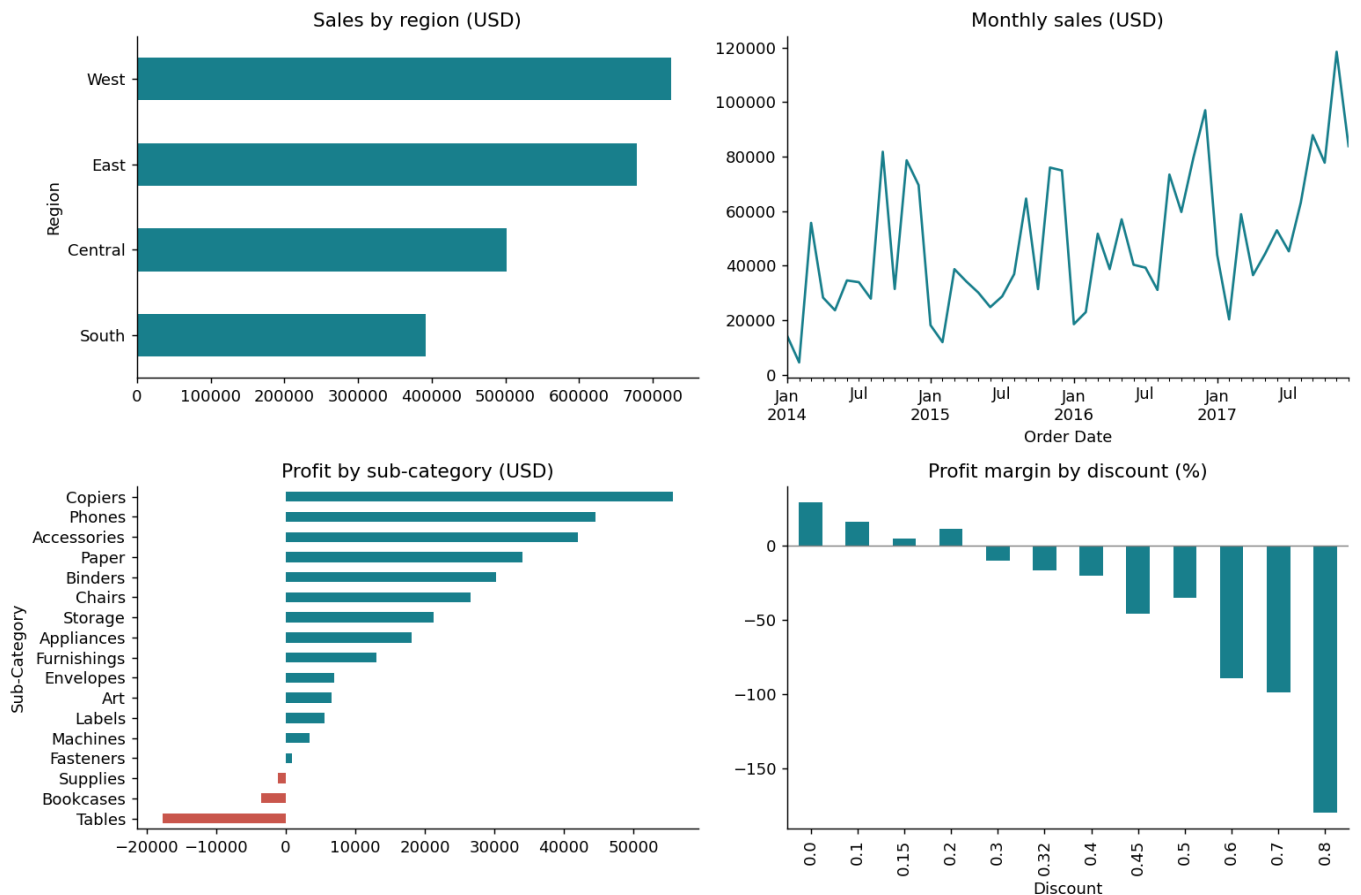
Sub-category	Sales	Profit	Margin
Tables	\$206,965.53	\$-17,725.48	-8.56%
Bookcases	\$114,880.00	\$-3,472.56	-3.02%
Supplies	\$46,673.54	\$-1,189.10	-2.55%

Technology is the strongest category by sales and profit. Furniture generates substantial sales but keeps little profit. The three loss-making sub-categories sum to a net loss of \$22,387.14. Review product costs, discounting, returns and fulfillment economics before expanding their volume.

SQL implementation

The SQLite database stores one row per order line. Ten executed queries cover KPIs, region, category, category losses, sub-category losses, monthly trends, discount groups, top products, order-to-ship time and integrity. The category-loss query correctly returns no rows. SQL totals reconcile with pandas; query result CSVs are included.

3. Trends and discount patterns



Year	Sales	Profit	Sales growth
2014	\$484,247.50	\$49,543.97	n/a
2015	\$470,532.51	\$61,618.60	-2.83%
2016	\$609,205.60	\$81,795.17	29.47%
2017	\$733,215.26	\$93,439.27	20.36%

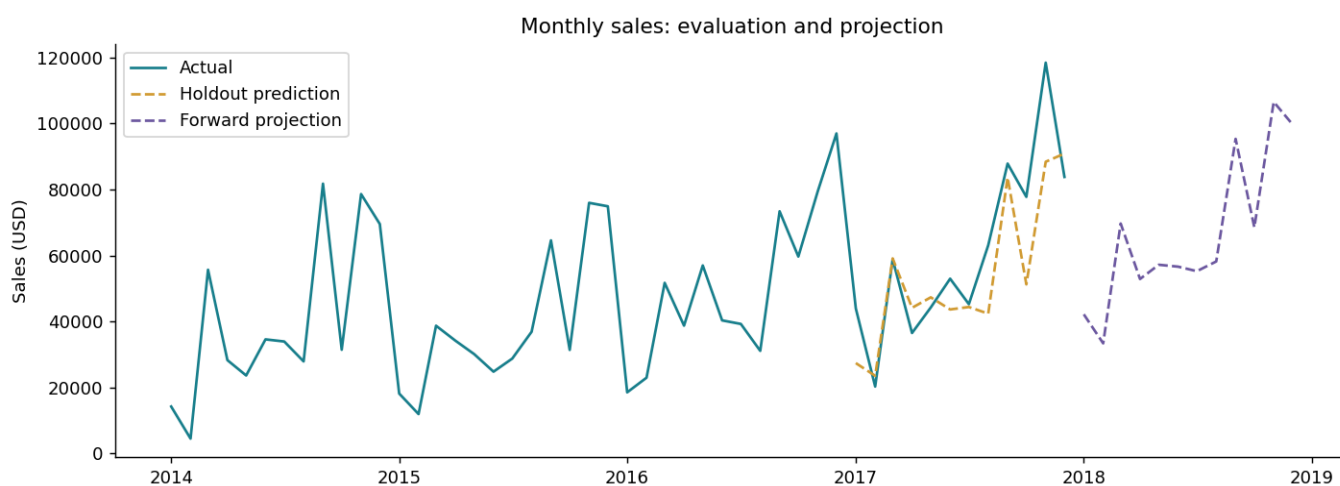
Sales show recurring late-year strength. November 2017 is the highest-sales month at \$118,447.83. Monthly totals fluctuate substantially, so a single straight time trend is insufficient; the forecast also includes month-of-year indicators.

Undiscounted lines earn a 29.51% aggregate margin. Lines discounted by 20% earn 11.82%; those discounted by 30% lose 10.05%. Product mix, geography and customer selection can confound these comparisons. Do not interpret them as a causal pricing experiment.

4. Machine learning and forecasting

Target: total monthly sales. Features: a time index plus 11 month-of-year indicators. Algorithm: scikit-learn LinearRegression with an intercept. Training uses January 2014-December 2016 (36 observations); testing uses January-December 2017 (12 observations). The test year is not used to fit the evaluated model. The baseline predicts each month using sales from the same month one year earlier.

Model	MAE (USD)	RMSE (USD)	R-squared	WAPE
Linear Regression	10,793.89	14,557.85	0.681	17.67%
Seasonal Naive	15,467.89	18,985.70	0.457	25.32%



MAE is average absolute monthly error. RMSE penalizes larger errors more strongly. R-squared compares prediction errors with variation around the test mean. WAPE divides total absolute error by total actual sales. The regression improves MAE by 30.22% over the baseline, but some peak months remain underpredicted.

After evaluation, the same specification is refitted on all 48 observed months to project 2018. Total projected sales are \$795,694.31. Monthly predictions and the historical holdout results are supplied as CSV files.

Limitations

Only four annual cycles are available. A single holdout year provides limited evidence of generalization. The model excludes promotions, prices, inventory and economic conditions and assumes stable seasonality plus linear growth. No calibrated prediction interval is supplied. Before operational use, run rolling-origin backtests, compare additional models and obtain recent data.

5. Dashboard and project use

Power BI report

The PBIP project defines Sales, Profit and Profit Margin cards, a monthly sales line chart, regional sales bars, category/sub-category profit bars, and Region/Category slicers. DAX calculates margin as $\text{DIVIDE}([\text{Total Profit}], [\text{Total Sales}])$. The cleaned rows are embedded in a compressed Power Query source, allowing the extracted project to move without breaking a source-file path.

Open `power_bi/Retail.pbip` in Power BI Desktop and click Refresh to load the model. If a PBIX is required, use Save As after reviewing the report. Report JSON is schema-validated; native Desktop refresh/rendering was not verified in this environment. The deliverable is PBIP project source, not a preloaded PBIX.

Recommended business actions

Investigate Tables and Bookcases first: both have negative profit within the low-margin Furniture category.

Track Central region margin alongside sales. Its 7.92% margin is below the 12.47% overall result.

Review high-discount transactions using product-level economics; test policy changes before broad rollout.

Plan stock with seasonal demand patterns, but keep uncertainty buffers and refresh the model using recent history.

Reproduce the analysis

1. Extract the full project folder. Open `Retail_Analysis.xlsx` for Excel results.
2. Install `python/requirements.txt` and run all cells in `python/Retail_Analysis.ipynb`. The submitted notebook already contains executed outputs.
3. Open `sql/retail.db` in a SQLite client and run `sql/analysis.sql`. To rebuild the database, run `python sql/import_data.py`.
4. Open the Power BI project, refresh and review. The included `source_refresh.pq` provides an optional CSV-based replacement query.

Source and verification

Data source: user-supplied `Sample - Superstore.csv`, preserved unchanged in `data/`. Calculations are independently reconciled between SQL and pandas. Excel KPI caches match those totals; formulas and native pivot definitions are structurally checked. The report and workbook previews were visually reviewed. Power BI project format reference: <https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview>